

Ejercicios Racso: Soluciones Teoricas para EDA Final

Nota de alcance

El grupo de Codeforces indicado requiere sesion activa o pertenencia al grupo, por lo que el scrape completo queda pendiente hasta que la pagina sea accesible desde el navegador. Esta hoja cubre los problemas identificados por las capturas y busquedas publicas. No se copian enunciados completos: cada problema tiene un resumen fiel, link oficial, teoria y solucion propia.

Mapa inicial

Tema	Problema	Link
Tree queries	Codeforces 375D - Tree and Queries	https://codeforces.com/problemset/problem/375/D
Strings	Codeforces 963D - Frequency of String	https://codeforces.com/problemset/problem/963/D
LCA/diferencia	Codeforces 191C - Fools and Roads	https://codeforces.com/problemset/problem/191/C
MST + LCA/RMQ	ICPC/Gym 101889I - Imperial roads	https://codeforces.com/gym/101889/problem/I
Dynamic graph	Yosupo - Dynamic Graph Vertex Add Component Sum	https://judge.yosupo.jp/problem/dynamic_graph_vertex_add_component_sum
Dynamic tree	Yosupo - Dynamic Tree Subtree Add Subtree Sum	https://judge.yosupo.jp/problem/dynamic_tree_subtree_add_subtree_sum
Dynamic tree connectivity	SPOJ DYNACON1	https://www.spoj.com/problems/DYNACON1/

1 Codeforces 375D - Tree and Queries

Tema

Subarboles, colores, consultas offline, Euler tour, DSU on tree o small-to-large.

Resumen del problema

Se tiene un arbol enraizado. Cada vertice tiene un color. Una consulta entrega un vertice v y un entero k , y pide cuantos colores aparecen al menos k veces dentro del subarbol de v .

Por que no basta una estructura simple

Una primera idea seria hacer un Euler tour: el subarbol de v se vuelve un intervalo $[\text{tin}(v), \text{tout}(v)]$. Eso transforma el problema a consultas de intervalo: “cuantos valores aparecen al menos k veces en este intervalo”. Pero k cambia por consulta y las frecuencias son por color. Un segment tree normal sobre el Euler tour no almacena facilmente la distribucion completa de frecuencias sin volverse pesado. Mo’s algorithm sobre arbol tambien podria funcionar offline, pero la solucion que se conecta mejor con las slides de HLD y small-to-large es DSU on tree.

Idea estructural

La tecnica DSU on tree procesa cada subarbol manteniendo una bolsa de colores. El truco es evitar reconstruir la bolsa desde cero para cada nodo.

Para un nodo u :

1. Se elige como hijo pesado al hijo con mayor tamaño de subarbol.
2. Se resuelven primero todos los hijos ligeros, pero al terminar se borra su informacion.
3. Se resuelve el hijo pesado y se conserva su informacion.
4. Se agregan los vertices de los hijos ligeros a la bolsa conservada.
5. En ese momento, la bolsa representa exactamente el subarbol de u , asi que se responden todas las consultas asociadas a u .
6. Si u no debe conservarse para su padre, se borra la informacion de todo su subarbol.

La estructura mantenida tiene dos niveles de conteo:

$$\text{freqColor}[c] = \text{numero de apariciones del color } c$$

y

$$\text{freqCount}[t] = \text{numero de colores que aparecen exactamente } t \text{ veces.}$$

Entonces la respuesta a “al menos k ” seria

$$\sum_{t \geq k} \text{freqCount}[t].$$

Para responder rapido, se puede mantener una estructura adicional sobre los valores t : un Fenwick/segment tree sobre frecuencias de frecuencias, o un arreglo de sufijos si las queries se procesan de forma conveniente. Conceptualmente, lo importante es que al agregar o quitar una ocurrencia de color c , su frecuencia cambia de old a $old + 1$ o $old - 1$, y por tanto solo se actualizan dos contadores de freqCount .

Correctitud

El invariante principal es: justo antes de responder las consultas del nodo u , la bolsa contiene exactamente los colores de los vertices del subarbol de u .

Se prueba por induccion sobre el arbol. Para las hojas, al agregar la hoja, la bolsa contiene su subarbol completo. Para un nodo interno, despues de resolver el hijo pesado y conservarlo, la bolsa contiene el subarbol pesado. Luego se agregan manualmente todos los vertices de cada hijo ligero, y finalmente se agrega u . Como los subarboles de los hijos son disjuntos y junto con u particionan el subarbol de u , la bolsa queda exacta. Las respuestas se calculan sobre esa bolsa, por lo que son correctas.

Complejidad

Cada vertice puede ser agregado varias veces, pero cada vez que se re-agrega es porque fue parte de un subarbol ligero que fue descartado. Al subir desde un vertice a la raiz, cada vez que se cruza una arista ligera, el tamaño del subarbol al menos se duplica. Por tanto hay $O(\log n)$ aristas ligeras en cualquier ruta. Cada vertice se procesa $O(\log n)$ veces en el peor analisis simple, por lo que la complejidad es $O(n \log n)$ mas el costo de responder consultas. En implementaciones classicas de DSU on tree con recorrido cuidadoso se obtiene muy buen rendimiento practico; con Fenwick sobre frecuencias, las actualizaciones cuestan $O(\log n)$.

Por que esta tecnica es examinable

Este problema obliga a reconocer tres ideas de EDA:

- Euler tour convierte subarbol en intervalo conceptual.
- Heavy/light explica por que descartar subarboles ligeros no explota.
- Small-to-large evita recomputar toda la informacion de cada subarbol.

2 Codeforces 963D - Frequency of String

Tema

Strings, muchas consultas de patrones, ocurrencias ordenadas, Aho-Corasick, suffix automaton/suffix array como alternativas conceptuales.

Resumen del problema

Hay un texto fijo S . Cada consulta trae un entero k y un patron P . Se pide la minima longitud de una subcadena de S que contenga al patron P al menos k veces. Si P aparece menos de k veces en todo S , la respuesta es -1 .

Observacion clave

Para un patron fijo P , supongamos que sus posiciones de ocurrencia en S , ordenadas de izquierda a derecha, son:

$$p_1 < p_2 < \dots < p_t.$$

Una subcadena que contiene al menos k ocurrencias puede elegirse para empezar en alguna ocurrencia p_i y terminar al final de alguna ocurrencia $p_j + |P| - 1$, con $j - i + 1 \geq k$. Para minimizar longitud, nunca conviene tomar mas de k ocurrencias si ya hay una ventana de k : para cada i , la mejor ventana que empieza en p_i y contiene k ocurrencias termina en $p_{i+k-1} + |P| - 1$. Entonces la respuesta para ese patron es:

$$\min_{1 \leq i \leq t-k+1} (p_{i+k-1} - p_i + |P|).$$

Como obtener las ocurrencias

Si solo hubiera una consulta, bastaria KMP/Z/Rabin-Karp. Pero hay muchas consultas y la suma de longitudes de patrones esta acotada. La estructura natural es Aho-Corasick:

1. Insertar todos los patrones en un trie.
2. Construir enlaces de fallo.
3. Recorrer el texto S una vez.
4. Cada vez que el automata cae en un estado, registrar que los patrones terminales de ese estado ocurren en la posicion actual.

El detalle teorico es que un estado puede representar varios patrones por enlaces terminales. Para evitar recorrer demasiados enlaces por caracter, se procesa el arbol de fallos offline: cada posicion del texto activa un estado, y cada patron recoge las posiciones de los estados en su subarbol de fallos. Otra forma valida en una respuesta a papel es decir: se usa Aho-Corasick con listas de salida optimizadas y se aprovecha que la suma total de patrones esta acotada.

Alternativa con suffix array

Tambien se puede razonar con suffix array. Para cada patron P , las ocurrencias de P son los sufijos que tienen prefijo P , y esos sufijos forman un intervalo contiguo en SA. Se encuentra ese intervalo con busqueda binaria. El problema restante es obtener las posiciones de esos sufijos ordenadas por posicion en el texto y luego aplicar la formula de ventana. Esto requiere una estructura de range reporting/range selection sobre el intervalo de SA, por lo que para el problema completo Aho-Corasick suele ser mas directo.

Correctitud

Primero, Aho-Corasick identifica todas las ocurrencias de todos los patrones porque cada camino del trie representa un patron y los enlaces de fallo simulan el mayor sufijo que todavia es prefijo de algun patron. Al recorrer S , cada vez que termina una ocurrencia de P , el estado actual tiene a P como patron terminal directo o via fallo.

Segundo, para un patron fijo, cualquier subcadena que contiene k ocurrencias contiene un primer inicio p_i y un k -esimo inicio posterior p_{i+k-1} . Para contener esa k -esima ocurrencia debe terminar al menos en $p_{i+k-1} + |P| - 1$. Por tanto su longitud es al menos $p_{i+k-1} - p_i + |P|$. La construccion que toma exactamente desde p_i hasta ese final alcanza esa cota. Minimizar sobre i da la respuesta optima.

Complejidad

Sea $N = |S|$ y L la suma de longitudes de patrones. Construir Aho-Corasick cuesta $O(L \cdot |\Sigma|)$ si se materializan transiciones, o $O(L)$ mas costo de mapas si se guardan sparse. Recorrer el texto cuesta $O(N)$ mas el costo de reportar ocurrencias. Para cada patron, si tiene t ocurrencias, calcular su respuesta para un k fijo cuesta $O(t)$. En el peor caso naive, muchas consultas sobre patrones con muchas ocurrencias pueden ser caras; por eso las soluciones competitivas agrupan patrones o usan tecnicas adicionales. En un examen teorico, la parte esencial es la reduccion a posiciones ordenadas y la formula de ventana.

Por que no usar solo suffix array

Suffix array es perfecto para existencia, conteo y listado de ocurrencias de un patron en texto fijo. Pero aqui no basta contar: se necesita la minima ventana por posiciones del texto. Eso introduce una segunda dimension: orden lexicografico en SA versus orden de aparicion en S . Aho-Corasick trabaja directamente en orden de texto y por eso encaja muy bien con la formula de ventanas consecutivas.

3 Codeforces 191C - Fools and Roads

Tema

LCA, diferencia en arbol, consultas de caminos, conteo por arista.

Resumen del problema

Se tiene un arbol. Luego se dan varias rutas entre pares de vertices. Para cada arista del arbol se quiere saber cuantas de esas rutas pasan por ella.

Idea principal: diferencia en arbol

En un arreglo lineal, para sumar 1 en un intervalo $[l, r]$, se puede hacer $diff[l]++$ y $diff[r+1]--$, y luego acumular. En un arbol, una ruta $u \rightsquigarrow v$ se maneja con LCA:

$$delta[u] += 1, \quad delta[v] += 1, \quad delta[lca(u, v)] -= 2.$$

Despues se hace un recorrido postorden. Para cada nodo x , se suma a su padre todo lo acumulado en su subarbol. El valor acumulado en x es la cantidad de rutas que pasan por la arista entre $parent(x)$ y x .

Por que funciona

Considera una sola ruta $u \rightsquigarrow v$ y sea $a = lca(u, v)$. La ruta sube desde u hasta a y baja desde a hasta v . Al poner $+1$ en u y $+1$ en v , la acumulacion hacia arriba marca todas las aristas de ambos lados. Pero si no restamos en a , la contribucion seguiria subiendo por encima de a , marcando aristas que no pertenecen a la ruta. Restar 2 en a cancela exactamente las dos contribuciones antes de que pasen al padre de a .

Para muchas rutas, se usa linealidad: las contribuciones de cada ruta se suman, y el postorden acumula todas a la vez.

Estructura de LCA

Para este problema basta binary lifting:

- Preprocesar profundidad y tabla de ancestros $up[x][j]$.
- Para $lca(u, v)$, primero igualar profundidades.
- Luego subir ambos vertices por potencias de dos hasta que sus padres coincidan.

Tiempo de preprocesamiento $O(n \log n)$, cada LCA en $O(\log n)$.

Complejidad

Con n vertices y q rutas:

$$O(n \log n + q \log n + n)$$

tiempo y $O(n \log n)$ espacio. Si se usa Euler tour + RMQ, se puede bajar LCA a $O(1)$ por consulta con preprocesamiento lineal o casi lineal, pero binary lifting es suficiente y mas facil de explicar.

Comparacion con HLD

HLD tambien podria sumar 1 sobre cada camino y luego consultar aristas, usando segment tree con lazy propagation. Eso daria $O(\log^2 n)$ por ruta. Pero como todas las actualizaciones son offline y todas son incrementos uniformes, la diferencia en arbol es mas simple y mas rapida: solo requiere una suma postorden.

4 Imperial roads - MST + LCA/RMQ

Tema

Minimum spanning tree, propiedad del ciclo, consultas max edge on path, LCA/RMQ.

Resumen del problema

Se tiene un grafo no dirigido, ponderado y conectado. Para cada consulta se fuerza que una carretera especifica pertenezca a la red final. Se pide el costo minimo de una red conectada que incluya esa carretera.

Solucion

Primero se construye un MST T del grafo, con peso total W . Luego se preprocesa T para contestar:

$$\text{maxEdge}(u, v) = \text{peso maximo de una arista en el camino entre } u \text{ y } v \text{ dentro de } T.$$

Para una carretera consultada $e = (u, v, w)$, se responde:

$$\text{ans}(e) = W + w - \text{maxEdge}(u, v).$$

Justificacion por ciclos

Al agregar $e = (u, v, w)$ al MST T , aparece exactamente un ciclo: la arista nueva mas el camino unico de u a v en T . Para obtener de nuevo un arbol generador que incluya e , hay que eliminar una arista de ese ciclo distinta o no necesariamente distinta de e segun si e ya estaba en el MST. Como e debe quedarse, conviene eliminar la arista de mayor peso del camino original. Eso minimiza el nuevo peso.

La propiedad del ciclo de MST dice que en un ciclo, una arista de peso maximo no es necesaria para algun MST. Aqui se usa de forma constructiva: entre todos los arboles que se obtienen agregando e y quitando una arista del camino, quitar la mas pesada da el menor costo.

Como responder maxEdge con LCA

Se enraiza el MST. Para cada nodo x y potencia 2^j , se guarda:

- el ancestro $up[x][j]$;
- el maximo peso de arista desde x hasta ese ancestro.

Para responder $\text{maxEdge}(u, v)$:

1. Igualar profundidades, acumulando maximos.
2. Subir ambos nodos desde potencias grandes a pequenas mientras sus ancestros difieran, acumulando maximos.
3. Finalmente considerar las dos aristas que conectan ambos nodos con el LCA.

Relacion con RMQ

Tambien puede verse como una consulta sobre caminos en arbol. Si la pregunta fuera solo LCA, Euler tour + RMQ resuelve minimo de profundidad. Para max edge on path, binary lifting augmentado es mas directo. HLD + segment tree tambien funciona: convierte el camino en $O(\log n)$ intervalos y consulta maximo.

Complejidad

Kruskal cuesta $O(m \log m)$. El preprocesamiento de LCA con maximos cuesta $O(n \log n)$. Cada consulta cuesta $O(\log n)$. El espacio es $O(n \log n)$.

Por que esta solucion es la esperada

El problema mezcla dos temas del final:

- MST: propiedad del ciclo y Kruskal.
- Static trees: responder informacion en el camino del MST con LCA/RMQ/HLD.

En papel, la parte que mas puntaje da es explicar por que reemplazar la arista maxima del camino es optimo.

5 Yosupo - Dynamic Graph Vertex Add Component Sum

Tema

Dynamic connectivity, component aggregate, offline dynamic graph, rollback DSU, segment tree over time.

Resumen del problema

Hay un grafo dinamico con pesos en vertices. Las operaciones incluyen agregar y eliminar aristas, sumar valor a un vertice y consultar la suma de valores de la componente conexas de un vertice.

Dos enfoques posibles

Online. Usar dynamic connectivity general con Euler-tour trees por niveles, como Holm-de Lichtenberg-Thorup. Es potente, pero complejo para implementar y explicar completo.

Offline. Si todas las operaciones se conocen antes de responder, cada arista existe en uno o varios intervalos de tiempo. Se construye un segment tree sobre el tiempo. Cada intervalo de vida de una arista se agrega a los nodos del segment tree que cubren ese intervalo. Luego se recorre el segment tree con un DSU con rollback. Este enfoque es mucho mas adecuado para una solucion teorica de concurso.

DSU con rollback

Un DSU normal comprime caminos, pero esa compresion destruye historia y complica deshacer. Para rollback se evita path compression y se usa union by size/rank. Cada union guarda en una pila que cambios hizo: padre cambiado, tamano cambiado y agregados de componente. Para volver a un estado anterior, se revierten esos cambios.

Para component sum, cada raiz mantiene:

$$sum[root] = \text{suma de pesos de los vertices en esa componente.}$$

Al unir componentes, se suman sus agregados. Al hacer rollback, se restauran.

Como manejar updates de vertices

La operacion de sumar a un vertice afecta a la suma de su componente actual. Hay dos formas teoricas de modelarlo:

1. Tratar el valor del vertice como un evento en el tiempo. En una hoja del segment tree temporal, antes de responder la query, se aplica el cambio si corresponde.
2. Mantener tambien rollback para valores: cuando se cambia el valor de un vertice, se actualiza la suma de su componente y se guarda el cambio para deshacerlo.

La idea esencial es que todo cambio que se hace al entrar a un nodo del arbol temporal debe poder deshacerse al salir.

Correctitud

Cada arista activa en un tiempo t fue insertada en exactamente los nodos del segment tree cuyos intervalos cubren sus intervalos de vida. Al llegar a la hoja t , el recorrido DFS ha aplicado todas y solo las aristas activas en ese tiempo. Por tanto, el DSU representa exactamente las componentes del grafo en el instante t . Como los agregados de suma se actualizan junto con las uniones y cambios de valor, la suma guardada en la raiz de la componente de v es exactamente la respuesta de la consulta.

Complejidad

Cada intervalo de vida de arista se inserta en $O(\log Q)$ nodos del segment tree temporal. Cada insercion causa una union con rollback de costo casi constante, normalmente $O(\log n)$ si se analiza altura sin compresion o $O(1)$ amortizado por union by size para encontrar raices con altura logaritmica. El total tipico se expresa como:

$$O((Q + M) \log Q \log N)$$

o mejor segun el analisis de find. El espacio es $O((Q + M) \log Q)$ para los intervalos distribuidos en el arbol temporal.

Por que no usar solo DSU normal

DSU normal no soporta eliminar aristas. La tecnica offline convierte cada eliminacion en un intervalo de vida y evita borrar: solo agrega al entrar a un intervalo y deshace al salir. Esa es la razon de usar rollback.

6 Yosupo - Dynamic Tree Subtree Add Subtree Sum

Tema

Euler-tour trees, arboles dinamicos, updates de subarbol, agregados con lazy propagation.

Resumen del problema

Se mantiene un arbol dinamico bajo operaciones de cortar y enlazar aristas. Tambien hay operaciones de sumar un valor a todos los vertices de un subarbol y consultar la suma de un subarbol.

Por que HLD estatico no basta

HLD clasico presupone un arbol fijo. Si se corta y enlaza, los tamanos de subarbol, padres, cadenas pesadas y posiciones Euler cambian. Recalcular HLD despues de cada operacion seria demasiado costoso. Se necesita una estructura dinamica.

Euler-tour tree

La idea de Euler-tour tree es representar cada arbol por la secuencia de su recorrido Euler, guardada en un balanced binary search tree implicito: treap, splay, AVL, etc. Las operaciones estructurales se vuelven operaciones sobre secuencias:

- split: partir la secuencia en dos;
- merge: concatenar dos secuencias;
- reroot: rotar la secuencia para cambiar la raiz representada;
- link: insertar la secuencia de un arbol dentro de la del otro;
- cut: separar el intervalo asociado a la arista eliminada.

Agregados de subarbol

Para subtree sum, se necesita que el subarbol de un vertice corresponda a un intervalo contiguo de la secuencia bajo una orientacion de raiz. Al mantener ocurrencias de entrada/salida o una variante de Euler tour orientada, el subarbol se aísla mediante splits. Sobre ese intervalo se puede:

- sumar lazy a todos sus vertices;
- leer su suma agregada;
- volver a unir las partes.

El BST implicito mantiene en cada nodo la suma del segmento, el tamaño y marcas lazy pendientes. No hace falta código para el examen; lo importante es explicar que las operaciones de secuencia preservan la representación del árbol y que los agregados se actualizan localmente al hacer split/merge.

Correctitud

El invariante es que cada componente árbol está representada por una secuencia Euler consistente. Link y cut modifican exactamente las apariciones que codifican la arista afectada, de modo que la secuencia resultante representa el nuevo bosque. Para una raíz fija, los vertices del subarbol de v aparecen en un bloque identificable por las ocurrencias de v . Al aislar ese bloque, cualquier lazy update aplicado al bloque afecta exactamente a los vertices del subarbol. La suma agregada del bloque es, por definición del invariante, la suma del subarbol.

Complejidad

Cada split o merge cuesta $O(\log n)$. Cada operación de alto nivel usa un número constante de splits/merges y operaciones lazy, por lo que link, cut, subtree add y subtree sum cuestan $O(\log n)$ amortizado o esperado según el BST usado.

Link-cut tree vs Euler-tour tree

Link-cut tree es excelente para agregados de camino: path sum, path max, path min. Euler-tour tree es mas natural para agregados de componente o subarbol, porque la representacion ya es una secuencia de recorrido. Por eso para este problema se prefiere Euler-tour tree.

7 SPOJ DYNACON1 - Dynamic Tree Connectivity

Tema

Conectividad dinamica en bosques: link, cut, connected.

Resumen del problema

Hay un bosque inicialmente formado por vertices aislados. Se reciben operaciones para enlazar dos vertices, cortar una arista existente y preguntar si dos vertices estan en la misma componente.

Enfoque online con Link-Cut Trees

Un link-cut tree mantiene un bosque dinamico en $O(\log n)$ amortizado por operacion. Sus operaciones conceptuales son:

- *find-root*: hallar la raiz representada de un vertice;
- *link*: unir dos arboles conectando una raiz con un vertice;
- *cut*: eliminar una arista;
- *connected*: comparar raices representadas.

Internamente usa preferred paths guardados en splay trees. La operacion clave es *access*, que reorganiza los preferred paths para exponer la ruta desde la raiz representada hasta el vertice.

Enfoque online con Euler-Tour Trees

Tambien se puede representar cada arbol por su Euler tour en un BST balanceado. Dos vertices estan conectados si sus ocurrencias pertenecen al mismo BST. Link y cut se realizan con splits/merges de secuencias. Para conectividad pura, Euler tour tree suele ser mas intuitivo que link-cut tree.

Enfoque offline con rollback DSU

Si todas las operaciones se conocen de antemano, se puede usar el mismo metodo de segment tree sobre tiempo:

1. Cada arista tiene intervalos de vida entre link y cut.
2. Se inserta cada intervalo en el segment tree temporal.
3. Se recorre con DSU rollback.
4. En cada hoja, connected se responde comparando representantes.

Este enfoque no es estrictamente online, pero para examenes es muy util porque demuestra como convertir eliminaciones en intervalos.

Correctitud

En link-cut/Euler-tour trees, el invariante es que la estructura representa exactamente el bosque actual. Link agrega una arista entre componentes distintas, cut elimina una arista existente, y connected pregunta si ambos vertices estan en la misma representacion. En rollback DSU, el invariante en la hoja temporal es que el DSU contiene exactamente las aristas activas en ese instante.

Complejidad

Link-cut tree: $O(\log n)$ amortizado por operacion. Euler-tour tree: $O(\log n)$ esperado/amortizado por split/merge, por tanto $O(\log n)$ por operacion. Offline rollback: cada intervalo se propaga a $O(\log q)$ nodos temporales; el costo total es casi $O(q \log q \log n)$ segun el costo de find sin compresion.

8 Comparacion final de tecnicas

Patron de problema	Estructura ideal	Razon	Complejidad tipica
Subarbol fijo con colores	DSU on tree	conserva hijo pesado y agrega ligeros	$O(n \log n)$
Muchas rutas offline en arbol	Diferencia + LCA	evita actualizar cada arista del camino	$O((n + q) \log n)$
Forzar arista en MST	MST + max edge path	propiedad del ciclo	$O(m \log m + q \log n)$
Patrones en texto fijo	Aho-Corasick o suffix array	procesa muchos patrones juntos	$O(S + L)$ base
Dynamic forest path aggregate	Link-cut tree	preferred paths	$O(\log n)$
Dynamic forest sub-tree/component	Euler-tour tree	arbol como secuencia dinamica	$O(\log n)$
Dynamic graph offline	Segment tree time + rollback DSU	elimina borrados via intervalos	$O(q \log q \log n)$

Checklist para responder a papel

1. Identifica si el arbol/grafos es estatico, dinamico online u offline.
2. Si es arbol estatico y camino: piensa LCA, HLD o diferencia.
3. Si es subarbol estatico: Euler tour o DSU on tree.
4. Si es string con texto fijo: suffix array/suffix automaton/Aho-Corasick.
5. Si hay link/cut online: link-cut tree para caminos, Euler-tour tree para componentes/subarboles.
6. Si hay deletes pero todo es offline: segment tree sobre tiempo + rollback.
7. Siempre cierra con correctitud y complejidad; eso vale mas que memorizar implementacion.